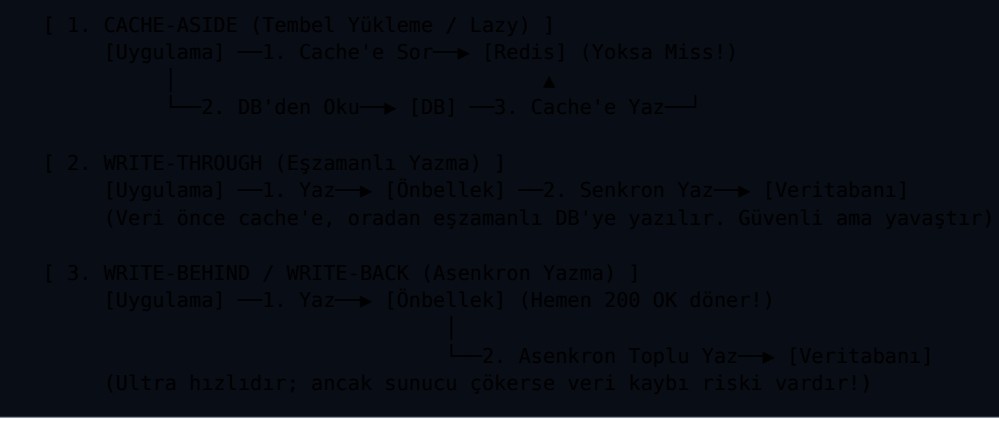
14

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Yazma Stratejileri: Cache-Aside, Write-Through ve Write-Back

Cache Writing Patterns: Cache-Aside, Write-Through & Write-Behind

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir not defteriniz (Cache) ve kalın bir arşiv defteriniz (DB) olsun: 1) Cache-Aside: Bir soru sorulduğunda önce not defterine bakarsınız; yoksa arşivden bulup not defterinize iliştirirsiniz. 2) Write-Through: Yeni bir bilgi geldiğinde aynı anda hem not defterine hem arşive yazıp öyle kafanızı kaldırırsınız (ikisi de hep günceldir ama yavaş yazarsınız). 3) Write-Back: Yeni bilgiyi hemen not defterine karalar ve 'Tamam!' dersiniz; akşama doğru arşiv defterine topluca geçirirsiniz. Ultra hızlıdır ama not defterini çay dökülüp kaybolursa bilgiler silinir!

🇬🇧 ENGLISH (ELI5)

Imagine a pocket notepad (Cache) and a heavy leather ledger (Database): 1) Cache-Aside: When asked a question, check your notepad; on a miss, look up the ledger and scribble the answer into your notepad for next time. 2) Write-Through: When recording an entry, write simultaneously to the notepad AND the ledger before responding (bulletproof consistency, higher write latency). 3) Write-Behind (Write-Back): Scribble into the notepad instantly and confirm completion; asynchronously batch-sync to the ledger later (extreme write speed, but if the notepad burns before sync, data vaporizes)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache-Aside (Lazy Loading): En yaygın modeldir (Memcached, Redis). Uygulama kodu önbellek ve veritabanı koordinasyonunu bizzat yönetir. Sadece talep edilen veriler önbelleğe alınır; dezavantajı: Cache Miss durumunda 3 ağ gidiş-gelişi (Cache oku -> DB oku -> Cache yaz). Veri güncellendiğinde DB yazılır ve cache `EVICT` (silinir) edilir. 2) Write-Through: Önbellek bir ara katmandır. Veri önce cache'e yazılır, cache senkron olarak DB'ye yazar. Tutarlılık yüksektir ancak yazma gecikmesi artar. 3) Write-Behind / Write-Back: Veri cache'e yazılır ve anında yanıt dönülür; arka plan worker'ları veriyi kuyruktan alıp DB'ye asenkron toplu (batch) yazar. Yoğun yazma trafiği olan sistemlerde (oyun skorları, sayaçlar, IoT sensör verileri) kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache-Aside: The application directly orchestrates cache and persistence. Misses fetch from DB and populate the cache. Updates write to DB and evict stale cache keys. 2) Write-Through: The cache acts as the main persistence abstraction. Writes update cache first, which synchronously flushes to DB before returning, guaranteeing strict read consistency at the penalty of elevated write latency. 3) Write-Behind (Write-Back): Writes hit the in-memory cache and return immediately (sub-millisecond ACK); background workers batch-drain dirty writes to DB asynchronously. Delivers unmatched write throughput for metrics, telemetry, and live view counters at the risk of transient data loss on node crashes.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Cache-Aside & Write-Behind Simülasyonu
import java.util.concurrent.*;

public class CachePatterns {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final BlockingQueue<String> writeBehindQueue = new LinkedBlockingQueue<>();

    // 1. Cache-Aside Okuma Mantığı
    public String getWithCacheAside(String key) {
        // Cache kontrolü
        String cached = cache.get(key);
        if (cached != null) return cached;

        // DB simülasyonu (Cache Miss)
        String dbValue = "DB_VAL_FOR_" + key;
        cache.put(key, dbValue);
        return dbValue;
    }

    // 2. Write-Behind (Asenkron Kuyruğa Atma)
    public void writeBehind(String key, String value) {
        cache.put(key, value); // RAM'e hemen yaz
        writeBehindQueue.offer(key + "=" + value); // DB worker'ına havale et
    }
}

void main() {
    var patterns = new CachePatterns();
    IO.println("Cache-Aside Değeri: " + patterns.getWithCacheAside("user:101"));
    patterns.writeBehind("score:99", "1500");
    IO.println("Write-Behind Başarılı (Bellek Güncellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Cache-Aside deseninde güncelleme yaparken önbelleğe yeni değeri YAZMAYIN (Cache Update)! Eski değeri SİLİN (Cache Eviction / Invalidation)! İki eşzamanlı yazma işlemi yarıştığında (Race Condition), önbellekte bayat veri kalabilir. En güvenli kural: DB'ye yaz, cache'i sil!

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

During mutations in Cache-Aside, do NOT update the cache in-place! DELETE/EVICT the key instead! In concurrent write races, in-place updates leave stale state. The golden rule: Commit to DB, then EVICT the cache key.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Cache-Aside okuma ağırlıklı (read-heavy) sistemlerin tartışmasız standart desenidir.
- Veri güncellendiğinde önbelleği güncellemek yerine anahtarı silmek (evict) yarış durumlarını önler.
- Write-Behind devasa yazma hacimlerini (yüksek QPS) veritabanına toplu aktarmak için idealdir.

🎯 Key Takeaways for the Interview

- Cache-Aside is the default pattern for read-heavy distributed architectures.
- Evicting cache keys upon DB mutations avoids dangerous concurrency race conditions.
- Write-Behind consolidates massive write bursts into efficient batch database flushes.